



Autobots 智能客服平台

全栈 AI 应用面试作战手册

Autobots · AP 平台 · WebGPT 爬虫 · 前端主导 · AI 辅助跨栈 · 源码深挖

适用方向：前端偏全栈 / AI 应用工程师 / 企业应用交付

基于 Autobots、agplatform 与 webgpt_server 源码快照及真实个人经历整理 | 2026-08-10

使用原则：前端与构建优化讲个人主导；Java/Python 讲 AI 辅助交付与可验证深度。

目录

按面试准备顺序编排；可使用 Word 导航窗格按标题快速跳转。

章节	你将掌握的内容
使用说明与事实边界	复习路径、源码快照、个人贡献分级、诚信表达
一页速览与项目话术	业务闭环、架构主线、30 秒到 3 分钟版本
业务与领域模型	用户、智能体、知识、会话、线索和任务状态
总体架构与服务边界	Vue、Gateway、Java 服务、FastAPI、AP、存储与外部依赖
跨项目架构：AP 与 WebGPT	控制面、业务面、爬虫执行面、配额预警与数据归属
三条核心调用链	WebGPT URL 摄入、RAG 流式问答、线索闭环
前端架构与工程化	三套应用、状态、流式 UI、构建发布、安全与测试
Java / Python / RAG	网关、适配层、异步边界、检索、Prompt 与评测
数据、安全与稳定性	状态机、幂等、最终一致性、权限、SSRF、可观测性
高频追问与系统设计	37 个技术问题、7 个场景题
STAR / 简历 / 行为面	真实主故事、AI 辅助表达、转型回答和投递定位
红线 / 自检 / 附录	不可夸大项、七天训练、口袋卡、模块地图与术语

使用说明与事实边界

这份手册不是让你背答案，而是建立一套可以根据岗位和追问深度自由切换的表达系统。先掌握“业务闭环 + 架构主线 + 两个真实故事”，再补技术细节。

三类事实与五种贡献标签

标记	含义	面试表达
源码事实	能从当前代码、SQL、脚本或测试中验证	“当前实现是……”
建议方案	当前未实现，是基于约束的演进设计	“如果继续演进，我会……”
待补数据	源码不能证明，需要监控、CI 或业务记录	“目前没有可靠口径，不能给出百分比。”
[主导]	负责方案并完成主要实现	“我负责……，关键取舍是……”
[参与]	承担清晰的一部分	“我参与……，主要负责……”
[AI 辅助]	AI 生成代码，你负责需求、审查、联调和验收	“这部分使用 AI 辅助实现，我能解释并对结果负责。”
[掌握]	不是你实现，但读过源码并参与联调	“不是我主导，我结合联调和源码理解了……”
[建议]	面向现状提出的改进	“当前限制是……，下一步会……”

诚信底线：项目中“存在某模块”不等于“我独立完成某模块”。最稳妥的句式是：“项目整体包含……；我实际负责……；我重点解决了……”。

个人事实卡

项目项	已确认事实
项目周期	2025.08–2026.08

项目项	已确认事实
执行团队	前端 1、Java 1、Python 1、测试 1、产品 1；另有 AP 项目统筹 2 人
正式岗位	前端工程师，团队中唯一前端
直接负责	app、app-manager、app-admin 全部前端应用和页面，以及前端构建发布
Java/Python	主要使用 AI 辅助生成；本人负责需求拆解、代码审查、联调、验证与排障
AP / 爬虫边界	AP 与 webgpt_server 是关联项目；可讲接口联调和源码理解，不表述为个人独立设计或从零实现
技术决策	前端生产构建优化：Arco 预构建复用 + 源码 hash 校验
线上问题	线索管理中产品与人员关联；具体根因和结果仍需补全
可公开规模	当前用户约 8 个；知识量、调用量和并发未形成可靠口径
AI 模型	主问答模型使用 DeepSeek；模型名称不是效果指标
效果指标	暂无可核验的耗时、错误率、成功率或人工成本数据

源码静态快照

维度	当前仓库快照	面试价值
前端	三套应用约 324 个 Vue/TS 文件、62,278 行；164 个 Vue 文件；42 个路由 path；59 个测试文件	多应用交付、复杂页面、工程化和测试讨论

维度	当前仓库快照	面试价值
Java	5 个模块约 317 个 main Java 文件、39,147 行；26 个 <code>@RestController</code> ；约 141 条 Mapping 声明	网关、业务服务、外部适配和数据层边界
Python	FastAPI 服务约 170 个 Python 文件、38,843 行；约 102 个路由装饰器	RAG、文档摄入、多渠道回调和 AI 服务编排
合计	三类主要源码约 14 万行	说明系统复杂度，不等于个人产出或线上规模

统计口径：排除前端 `dist`、Python `.venv/__pycache__`，按当前文件夹静态快照计算。行数、文件数和接口声明不能替代用户量、QPS、成功率或个人贡献。

范围口径：上表只统计 Autobots 主仓库。`../agplateform` 和 `../webgpt_server` 用于核对跨系统边界，不计入个人代码量，也不作为个人产出证明。

推荐复习路径

- 1. 面试前 30 分钟：一页速览、90 秒介绍、主 STAR、红线、口袋卡。
- 2. 面试前 1 天：三条调用链、37 个问题、两个 STAR，至少口述两轮。
- 3. 面试前 3-7 天：补真实数字，按七天训练计划做系统设计和模拟面试。

一页速览与项目话术

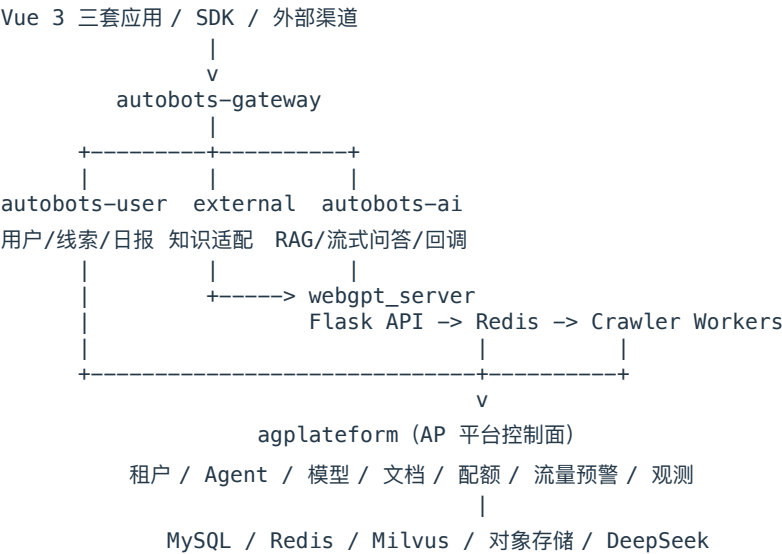
一句话定位

Autobots 是面向企业的智能客服与 AI 应用平台，把“智能体配置、知识摄入、RAG 问答、多渠道接入、线索沉淀和运营管理”串成一条业务闭环。

业务主链路

- 企业配置智能体
- > 上传文档 / 导入网页 / 配置问答对
 - > 异步解析、切分、Embedding、写入向量库
 - > 用户从 Web 或外部渠道提问
 - > 按身份和智能体范围计算可检索知识
 - > 召回、重排、Prompt 组装、DeepSeek 生成
 - > POST 流式返回前端
 - > 会话落库、线索抽取、通知和运营统计

架构主线



最值得深挖的四个点

1. **【主导】** 三套 Vue 应用全量交付：C 端聊天、企业管理端、平台管理端。

2. [主导] 生产构建优化：预构建产物复用、SHA-256 manifest、环境 mode 和干净容器发布。
3. [AI 辅助/掌握] AP 跨系统集成：Agent、模型、租户配额、流量预警及失败降级。
4. [AI 辅助/掌握] WebGPT URL 知识摄入：Redis 任务、抓取策略、父子文档、`task_id` 回调。
5. [参与/待补] 线索管理产品与人员关联：业务规则、字段契约、回归验证和线上恢复。

一句话取舍

当前架构用 AP 承接平台级控制面，用 Autobots 承接智能客服业务和 RAG 编排，用 WebGPT 承接资源特征独立的网页抓取，前端用三套工作台承接配置与交互。在约 8 个用户的阶段优先交付闭环，下一步应补齐可靠事件、统一可观测、RAG 评测和跨服务授权。

30 秒版本

我在 Autobots 项目担任唯一前端，负责 Vue 3 + TypeScript 的 C 端聊天、企业管理端、平台管理端和生产构建发布。系统由 Autobots 业务服务、AP 平台控制面和 WebGPT 爬虫协作，把知识导入、RAG 问答、多渠道接入和线索运营串成闭环。我主导过 Arco 预构建和源码 hash 校验；Java/Python 与跨系统部分主要使用 AI 辅助，我负责需求拆解、联调、验证和问题定位，并如实区分主导经验与辅助经验。

90 秒版本

Autobots 是一个企业智能客服平台。它不是孤立服务：AP 负责租户、Agent、模型、配额和流量观测等平台能力；Autobots 负责客服业务、知识编排、RAG、会话和线索；WebGPT 负责网页抓取。企业导入 URL 后，WebGPT 通过 Redis 分发任务并把网页写成父子文档，再用真实 `task_id` 回调知识生效；用户提问时，Autobots 按服务端知识范围召回证据，交给 DeepSeek 生成并流式返回。

我在项目中是唯一前端，负责 `app`、`app-manager`、`app-admin` 三套应用，包括聊天、智能体、知识库、对话、线索、数据看板和运营配置。最能代表我个人能力的是生产构建优化：旧流程在干净容器里每次都重建 Arco，我把它拆成组件库预构建和日常业务构建，用源码 SHA-256 与

`.build-manifest.json` 校验源码和产物一致性，使普通业务发版跳过整套组件库重建，同时保留失败保护。

为了向全栈转型，我也通过 AI 辅助进入 Java/Python 和关联项目代码，能够从前端 API 跟到 Gateway、Spring Controller/Service、FastAPI Route/Service/Repository、WebGPT Worker、Redis、AP 内部接口、MySQL 和 Milvus，解释状态、异常、权限和最终一致性。我的优势不是假装有多年纯后端经验，而是用成熟前端能力承担端到端交付，并逐步把 AI 辅助开发提升为独立后端设计与维护能力。

3 分钟展开顺序

1. 30 秒讲业务闭环和用户价值。
2. 30 秒画出前端、Autobots、AP、WebGPT 和存储边界。
3. 60 秒讲生产构建优化 STAR，明确个人主导。
4. 40 秒讲 WebGPT 知识摄入、AP 集成与问答链路，明确后端为 AI 辅助。
5. 20 秒讲现有不足：指标、可靠任务、RAG 评测和授权。

业务与领域模型

核心用户与业务对象

业务对象	作用	关键问题
企业 / 用户	隔离账号、配置和数据	身份、租户、角色如何传递和校验
智能体 Agent	聚合模型、Prompt、知识、渠道和展示配置	发布状态、版本、知识范围、渠道密钥
知识文档	文档、URL、问答对及其解析状态	异步生效、重复导入、失败重试
知识任务	一次爬取、解析或向量化执行	<code>task_id</code> 、状态机、幂等、可观测
会话 / 消息	保存提问、回答和上下文	流式中断、顺序、记忆窗口、隐私
线索 Lead	从表单或对话中沉淀的业务机会	归属、产品与人员关联、导出与权限
渠道 Channel	Web、微信、企微、飞书、钉钉等入口	回调鉴权、签名、重试和消息幂等

三个必须讲清的标识

- `trace_id` = 一次跨服务请求，用于日志关联
- `task_id` = 一次知识处理任务或知识分组，用于状态与检索关联
- `session_id` = 一段连续对话，用于消息和上下文关联

`task_id` 是关联键，不是授权凭证。用户能访问哪些知识任务必须由服务端根据用户、租户、智能体和知识关系计算，不能相信前端传入的任意列表。

知识状态

建议把知识处理理解成显式状态机：

```

submitted -> crawling -> parsing -> embedding -> effective
      |           |           |           |
      +-----+-----+-----+-----> failed

failed -> retrying -> crawling/parsing/embedding
effective -> refreshing -> effective/failed

```

提交接口成功，只说明系统接收了任务；爬取成功，只说明拿到了内容；只有向量写入并通过检索可见性校验后，才能称为“知识已生效”。

线索闭环

线索可能来自公开表单、预约、对话内容或外部渠道。线索价值不在“建一张表”，而在于：来源可追踪、产品与人员关系正确、权限范围明确、重复线索可处理、跟进状态可审计、导出和通知可控。

总体架构与服务边界

模块职责

模块	责任	面试关键词
<code>autobots-frontend/app</code>	C 端聊天、登录和用户体验	流式 UI、会话、异常和取消
<code>app-manager</code>	智能体、知识、对话、线索、看板、运营配置	复杂表单、权限、状态驱动、API 契约
<code>app-admin</code>	用户、企业、合同和平台数据	管理台边界、角色和审计
<code>autobots-gateway</code>	统一入口、JWT/Cookie 鉴权、路由和 CORS	信任边界、上下文传播、超时
<code>autobots-user</code>	用户、线索、日报和流量预警	传统业务、事务、定时任务
<code>autobots-external</code>	知识、文件、网页爬取和外部服务适配	Adapter、超时、重试、错误映射
<code>autobots-dao/common</code>	实体、Mapper 和通用能力	数据模型、统一响应、异常
<code>autobots-ai</code>	文档摄入、RAG、流式问答、渠道回调	FastAPI、Milvus、Embedding、DeepSeek
<code>../agplateform</code> (AP)	租户、Agent、模型、文档、配额、流量预警和平台观测	控制面、多租户、内部 API、配置归属
<code>../webgpt_server</code>	URL 爬取 API、Redis 任务调度、抓取 Worker、父子文档推送	异步任务、策略选择、幂等、SSRF、回调

为什么 Java 与 Python 共存

Java 适合承接账号、权限、线索、报表、调度和外部系统适配等稳定企业业务；Python 适合快速集成 LangChain、Embedding、Reranker、向量库和模型 SDK。边界应按变化率 and 能力生态划分，而不是按“哪种语言更高级”。

共存的代价是跨服务契约、错误模型、身份传播、链路追踪和一致性更复杂，因此需要统一 `trace_id`、超时预算、错误码、接口文档和契约测试。

请求生命周期

Browser request

- > Gateway 认证身份、注入受信用户/租户上下文
- > 路由到 user / external / ai
- > Controller/Route 校验请求 DTO
- > Service 编排业务与外部依赖
- > Repository/Mapper 访问 MySQL、Milvus、对象存储
- > 统一响应或 StreamingResponse
- > 日志使用 trace_id/task_id/session_id 关联

当前合理性与演进边界

约 8 个用户不需要先假设高并发。当前更重要的是正确性、可排障和交付效率。随着任务量增长，优先拆的是资源特征不同、故障影响大、需要独立扩容的异步摄入与模型调用 Worker，而不是为了“微服务”继续拆 CRUD。

跨项目架构：agplatform、AP 与 WebGPT

三个系统为什么要分开

面试时不要把 AP、Autobots 和爬虫说成三个随意拆分的 CRUD 项目。它们分别承担不同变化率、资源模型和故障半径：

系统	架构角色	核心责任	不应承担
agplatform / AP	平台控制面	租户、Agent 生命周期、模型供应商、文档平台、配额、流量预警、观测	Autobots 特有的线索和客服页面逻辑
Autobots	AI 客服业务面	用户与运营、知识编排、RAG 问答、会话、线索、渠道和日报	通用 Agent 平台能力的重复建设
webgpt_server	网页采集执行面	URL 校验、任务调度、抓取、正文清洗、链接发现、父子文档推送	用户权限、Agent 配置和 RAG 在线生成

这种拆分的收益是独立演进和资源隔离：爬虫是 I/O 密集且面对不可信网页，模型调用受 Provider 配额影响，企业业务更强调事务和权限。代价是内部认证、契约、超时、幂等、状态同步和跨系统排障都更复杂。

AP 的技术边界

当前源码快照中，AP 是多技术栈平台：Java 21 / Spring Boot 3.5 承担核心服务，React 18 + TypeScript 承担管理前端，Go 承担编排能力，Rust 承担部分运行时。面试不需要逐语言背框架，而要讲清它作为控制面的数据所有权。

AP 能力	Autobots 如何使用	数据归属与关键约束
租户与配额	查询/更新 <code>/internal/tenant/{tenantId}/quotas</code> ；充值退款写 <code>quota ledger</code>	AP 是配额真相源；ledger 需要业务幂等键
Agent 生命周期	查询、创建、更新、删除、下线、发布 Agent	AP 保存平台 Agent 配置，Autobots 做业务映射与页面交互
模型供应商	调用 <code>/api/v1/mcp/llm-providers/models/unified</code> 获取可用模型	<code>sourceId</code> 应来自 AP 动态配置，不能在环境变量中硬编码具体模型记录
推荐问题	调用 Agent recommendations 接口，按 Agent/会话生成推荐	推荐失败不应破坏主问答，可降级为空列表
流量预警	配置、开关、当前用量、阈值检查、历史和测试通知	AP 持有配置与平台用量；Autobots 补充本地业务指标并触发通知
文档平台	WebGPT 将根文档和子页面推送到 AP 文档接口	文档 ID 是父子关系主键； <code>task_id</code> 是一次爬取的跨系统关联键

AP 配额与流量预警链路

- 一次问答完成
 - > Autobots AI 获取/归一化 Token 与调用用量
 - > 调用 AP `/internal/traffic-alert/check`
 - > AP 返回配置、配额、阈值判断或当前用量
 - > `autobots-user TrafficAlertAutoService` 合并本地业务数据
 - > 按租户 + 月份 + 指标 + 阈值做 Redis 去重
 - > 多渠道通知并记录预警历史

关键取舍有三点：

1. 主链路优先可用。Python 侧对预警检查采用 fail-open：AP 预警异常只记录日志，不阻断聊天。Java 侧保留本地阈值判断作为降级。这里的 fail-open 只适用于旁路预警，不能用于认证、扣费或越权判断。

2. 配额字段必须有唯一语义。monthlyTokens 是月 Token 上限，monthlyApiCalls 是月对话轮次上限；maxConversationTurns 是单会话最大轮次，不能混为一个指标。同步中的 -1 表示无限，不等于缺失或 0。
3. 日报按字段归属聚合。会话数、独立 IP、平均轮次、消息、首 Token、线索和热点主题由 Autobots 计算；配额与流量用量来自 AP。Autobots 可以读取和展示，不能伪造或覆盖 AP 的真相值。

AP 集成如何保证可靠

风险	当前处理思路	更完整的演进
AP 短暂不可用	超时、日志、非关键链路 fail-open、Java 本地阈值回退	熔断、半开探测、缓存最后一次配置并标注时间
重复充值/退款	quota ledger 记录业务请求	tenant_id + business_order_id + operation 唯一约束
字段别名漂移	客户端归一化多种用量字段	OpenAPI/契约测试，最终收敛唯一字段名
租户 Header 被伪造	内部 Token 与租户 Header 一起传递	服务间身份、网络隔离、签名或 mTLS；AP 再校验租户权限
双方状态不一致	明确 AP 为平台配置真相源	版本号/ETag、Outbox、重放和对账任务

WebGPT 的组件与任务模型

- POST /crawl
 - > 校验并规范化 URL、生成 task_id
 - > 在 AP 创建根文档占位，缓存 parent_doc_id (TTL 24h)
 - > Redis Hash 保存任务元数据
 - > 任务专属优先队列 + 全局桶队列公平调度
 - > Crawler Worker 领取任务并并发抓取
 - > Markdown 清洗、链接发现、URL 去重
 - > 根文档更新 / 子页面携带 parentId 写入 AP
 - > 根任务终态只更新一次
 - > 回调 Autobots AI knowledge-effectiveness

对外接口的语义：

接口	语义	面试注意点
POST /crawl	接收 URL、深度、最大页数、允许域名等，返回 task_id	返回成功只是入队，不代表知识可检索
GET /status/{task_id}	读取 Redis 中 total/completed/failed/running 等进度	当前 Redis 状态缺失会被解释成 completed，存在语义歧义
POST /cancel/{task_id}	清理任务和待处理队列	删除状态不等于已运行抓取一定立刻停止，需要协作式取消

任务状态包含 pending/running/completed/failed/cancelled。任务对象保存 URL、深度、最大深度、最大页数、允许域名、父任务、重试次数和 Worker 等上下文；每个会话用规范化 URL visited set 去重，并设置 TTL，防止无限扩张。

为什么有三种抓取策略

WebGPT 先用 Requests 抓取。如果页面已发现足够多的链接，就直接使用轻量结果；如果链接较少，则并行比较 Requests、SessionPage 和 Playwright 的链接数量与可用内容，选择更合适的策略并把结果缓存到 Redis。

策略	优势	代价	适用页面
Requests	快、资源低、并发成本小	不能执行 JavaScript	静态 HTML、服务端渲染页面
SessionPage	能保留会话状态，成本低于完整浏览器	动态兼容性仍有限	Cookie/会话相关但交互不复杂的页面
Playwright	能执行 JS，适合动态 DOM	浏览器启动重、内存高、故障面大	SPA、客户端渲染和需要真实浏览器的页面

这里体现的是“渐进增强”：先走便宜路径，证据不足时再升级重策略。继续演进时应复用浏览器池和 Context，按域名限制并发，记录策略命中率、抓取成功率、单页耗时和内存，而不是每页无条件启动完整浏览器。

父子文档与知识生效

根页面在任务入队前先创建占位文档，是为了解决子页面先完成却找不到 `parentId` 的竞态。Worker 抓到根页面正文后更新占位文档；子页面等待并读取根文档 ID，写入时携带 `parentId`。根任务结束时通过 Redis `SETNX` 类机制只更新一次 `crawlStatus`。

知识生效指标使用爬虫实际返回的 `task_id`：Autobots external 先创建 pending 事件，WebGPT 根文档写入成功或失败后回调 `autobots-ai/server/ingest_route.py` 的 `knowledge-effectiveness` 接口，最终把事件标记为 `effective` 或 `failed`。回调遇到 pending 尚未可见的短暂竞态时可收到 `409` 并有限重试；旧的 `distributed_crawler_data` 摄入路径不是这条指标链路的依据。

WebGPT 的高风险点

- 1. **SSRF**：允许域名只能缩小范围，仍需在初始解析和每次重定向后检查 DNS、私网、loopback、link-local 和云 metadata 地址。
- 2. **凭证进入队列**：当前任务需要调用 AP 的凭证上下文；长期方案应在 Redis 只保存短期、最小权限的凭证引用，避免保存可复用原始 Token。

3. **TLS 校验：** 外部请求不能把关闭证书校验当成生产默认值；应使用可信 CA、域名校验和受控代理。
4. **状态丢失：** Redis 清理或过期后不能把“不存在”直接等同“成功”；终态应持久化到任务表，状态接口区分 `not_found/expired/completed/cancelled`。
5. **回调可靠性：** 有限重试后仍可能丢失终态；关键指标应通过 Outbox、持久化重试或死信队列补偿，并保证消费者幂等。
6. **取消语义：** 清队列只能阻止未执行任务；运行中的 HTTP/浏览器请求需要取消标志、超时和 Worker 协作检查。

你的安全表达边界

推荐表达：

AP 和 WebGPT 不是我个人从零实现的模块。我作为 Autobots 唯一前端，在页面、接口和跨系统联调中接触到它们；后端代码主要使用 AI 辅助。我能够沿真实源码解释 Agent/模型/配额归属、爬虫任务状态、父子文档、回调幂等和失败降级，也能给出测试与演进方案，但不会把平台架构包装成个人主导成果。

如果面试官问“那这算不算全栈经验”，回答重点是：你能否独立定位契约问题、读懂状态和数据流、验证异常分支，并逐步承担一个有测试和上线责任的小型后端纵切，而不是是否手写了每一行。

核心链路一：URL 知识摄入

用户视角

企业用户在管理端输入 URL，可配置刷新策略；系统提交爬取任务，页面展示处理中、已生效或失败，并能查看子页面和刷新设置。

端到端链路

```

UrlUploadModal.vue
-> POST /external/knowledge/from-url
-> Gateway 注入身份
-> autobots-external KnowledgeServiceImpl 调用爬虫
-> webgpt_server POST /crawl
-> 生成真实 crawler task_id, 先在 AP 创建根文档占位
-> Redis 保存任务、visited set、优先队列和 parent_doc_id
-> Crawler Worker 选择 Requests / SessionPage / Playwright
-> 清洗 Markdown、发现子链接、按域名/深度/页数继续入队
-> 根文档更新、子页面携带 parentId 写入 AP 文档接口
-> WebGPT 回调 autobots-ai /knowledge-effectiveness
-> 生效事件按 task_id 更新 effective_at/status/document_id
-> RAG 通过 AP 文档/摄入链路获得可检索知识
-> 前端列表展示最终状态
  
```

这条链路里有两个看似相同但不能混用的 ID：Autobots 提交时可能带请求任务标识，真正用于知识生效跟踪的必须是 WebGPT 响应中的 crawler `task_id`；AP 返回的 `document_id` 用于父子文档关系和文档操作。`task_id` 关联一次任务，`document_id` 标识一条文档资源。

为什么异步

爬虫、下载、解析和向量写入都有不稳定耗时和多个失败点，长时间占用 HTTP 连接会受到网关超时和用户刷新影响。先返回任务标识，再异步处理，可以分离“请求已接收”和“知识已生效”，代价是必须设计状态、幂等、重试、补偿和可观测。

竞态与幂等

当前源码中存在“爬虫回调可能先于 pending 生效事件可见”的竞态，回调找不到记录时用 409 表达临时状态冲突并允许有限重试。Autobots external 在拿到 WebGPT 的真实 `task_id` 后创建 pending 事件；若网页很小、Worker 很快，回调仍可能抢先。更稳的设计是由任务接收方先持久化任务事件，再发布执行消息，或使用可按 `task_id` 幂等 upsert 的状态记录与 Outbox。

重复提交可用以下幂等键之一：

1. 客户端生成 `idempotency_key`，服务端保存唯一索引。
2. 使用 `tenant_id + agent_id + normalized_url + content_version` 形成业务键。
3. WebGPT visited set 按规范化 URL 去重，限制同一任务重复抓取。
4. AP 文档写入使用业务键或幂等请求，避免 Worker 重试产生重复子文档。
5. 回调按 `task_id + event_type + provider_event_id` 去重，终态不能倒退。

根文档占位解决什么问题

如果根页面和子页面并行处理，子页面可能先抓完，但此时 AP 还没有根文档 ID，`parentId` 无法填写。WebGPT 在任务入队前先创建根文档占位，并把 ID 放入 Redis；Worker 后续更新根正文，子页面读取该 ID 建立层级。这个方案缩小了竞态窗口，但仍要处理占位创建失败、TTL 到期、根更新失败和孤儿子文档。

状态和取消的正确语义

```
pending -> running -> completed
                -> failed
                -> cancelled
```

`completed` 应表示所有必要页面已经处理并且终态已持久化，而不仅是 Redis 中找不到任务。

`cancelled` 需要同时阻止未执行任务、通知运行中的 Worker、终止可取消请求并记录终态。面试时主动指出“状态缺失被当作完成”和“删队列不等于停止运行任务”，会比只背状态枚举更有深度。

排障顺序

1. 前端 `/external/knowledge/from-url` 是否成功，拿到的是不是 WebGPT 实际返回的 `task_id`。
2. external 调用 `/crawl` 是否超时或映射错误，pending 生效事件是否以同一 `task_id` 创建。
3. WebGPT Redis 任务、visited set、队列和进度是否存在，任务在哪个 Worker、使用哪种抓取策略。
4. 根文档占位是否成功，`parent_doc_id` 是否仍在 TTL 内，子页面是否因等待超时被跳过。
5. AP 文档写入是否成功，根/子文档层级、`crawlStatus`、租户和 Agent 归属是否正确。
6. knowledge-effectiveness 回调是否成功；若为 409，是否重试并最终进入 effective/failed。
7. RAG 侧知识是否已解析和可检索，Milvus metadata 与服务端 `task_ids` 过滤是否一致。
8. 前端轮询是否把抓取完成、解析完成和知识生效混成一个状态。

核心链路二：RAG 流式问答

在线流程

用户提问

- > 服务端根据身份/智能体计算可访问 task_ids
- > Query Rewrite (按需)
- > Embedding
- > Milvus 按 task_ids/tenant/agent 过滤召回
- > Rerank 与阈值过滤
- > Prompt = 系统约束 + 历史 + 证据 + 当前问题
- > DeepSeek 流式生成
- > FastAPI StreamingResponse
- > 前端 fetch + reader + TextDecoder 增量展示
- > 结束后保存会话、线索和统计

为什么使用 POST + 流式响应

请求体需要携带问题、会话、智能体和上下文，POST 更适合结构化输入。浏览器原生 `EventSource` 主要面向 GET，因此前端用 `fetch` 读取 `ReadableStream`。这样可以保留 POST 语义，但断线重连、取消、事件解析和错误恢复需要自行实现。

前端流式解析要点

1. `TextDecoder` 使用流式模式，不能假设一个网络 chunk 就是一个完整字符或事件。
2. 保留未完成 buffer，按协议分隔符解析完整事件。
3. 区分 token、error、finished 等事件类型。
4. 使用 `AbortController` 取消请求；只停止 UI 更新并不会停止服务端生成。
5. 限制高频响应引起的渲染次数，可按帧或小时间窗批量刷新。
6. 中断时保留已生成内容，并明确“已停止/连接中断”，避免伪装成功。

如何减少幻觉

RAG 不是“用了向量库就不会幻觉”。需要同时控制：知识权限、召回质量、重排阈值、Prompt 引用规则、低证据拒答、上下文长度和评测集。回答应携带可追踪证据，证据不足时明确拒答或建议人工确认。

RAG 评测

层级	指标	说明
检索	Recall@K、MRR、命中率	正确证据能否进入候选和靠前排列
生成	事实一致性、答案相关性、引用正确性	回答是否忠于证据且解决问题
拒答	精确率、召回率	无证据时是否拒答，有证据时是否误拒答
体验	首 Token、完整耗时、流中断率	用户是否尽快看到稳定输出
业务	解决率、转人工率、线索有效率	系统是否真正完成业务目标

核心链路三：线索与运营闭环

线索来源与处理

线索可能来自公开表单、预约、会话抽取或渠道回调。创建时要保留来源、智能体、会话、产品、负责人、联系方式和时间等上下文，之后才有条件做分配、筛选、导出、通知和统计。

产品与人员关联问题 STAR 骨架

这个案例目前信息不足，文档只保留骨架，面试前必须补真实记录。

项目	必须补齐的事实
故障表现	是显示、筛选、归属、保存还是统计错误？
影响范围	哪些角色和多少条数据受到影响？是否阻断跟进？
根因	前端字段映射、接口契约、数据库关系还是产品规则不清？
处理动作	改了哪些页面、API、字段和回归场景？
验证	如何构造数据，如何确认旧数据和新增数据都正确？
结果	恢复了什么能力，有无工单、用户确认或耗时变化？

安全表达：我处理过线索管理中产品与人员关联的线上问题，主要负责前端链路、字段核对和跨角色联调；具体根因和结果以真实记录为准，不补写未经核实的收益。

线上问题处理框架

1. 先止损：确认影响、关闭错误入口或回退变更。
2. 保留证据：请求参数、响应、用户角色、记录 ID、时间和版本。
3. 缩小范围：区分展示错误、请求错误、持久化错误和业务规则冲突。
4. 修复契约：字段命名、必填性、枚举、关联 ID 与展示名称分开。

5. 回归矩阵：新增/编辑/筛选/导出，管理员/普通用户，旧数据/新数据。
6. 上线验证：真实角色检查、日志确认、监控观察和产品验收。

前端架构与复杂交互

三套应用如何分工

app 偏 C 端聊天体验；app-manager 是企业日常工作台，业务最复杂；app-admin 是平台管理端。三者共享 Vue 3、TypeScript、Vite、Pinia、Router、Axios 与 Arco Design，但权限、路由和发布目标不同。

页面分层

- View/Page
 - > 业务组件与弹窗
 - > composable/store 管理跨组件状态
 - > api 模块封装 URL、DTO 和响应
 - > request 层统一鉴权、错误与取消

页面不应直接散落请求路径和错误处理。复杂业务状态优先用显式枚举和派生状态，避免多个布尔值组合出不可能状态。

知识状态驱动 UI

后端状态	UI 行为
submitted/crawling/parsing/embedding	展示处理中、允许刷新，限制重复提交
effective	可用于问答，显示生效时间和范围
failed	展示失败阶段和可操作重试，不只显示“失败”
refreshing	保留旧版本可用性，同时展示更新中

TypeScript 不能替代后端校验

TypeScript 只在编译期约束可信源码，运行时请求、旧版本客户端和恶意调用都可能绕过。前端类型用于开发效率；后端 DTO/Pydantic 用于信任边界；数据库约束用于最终数据不变量，三者各司其职。

前端安全

1. Token/Cookie 策略要结合 XSS、CSRF 和跨域约束，优先 HttpOnly Cookie 或短期 Token。
2. 不在 `localStorage` 保存长期敏感凭据或完整隐私数据。
3. 富文本、Markdown、链接和文件名都要按输出上下文转义或白名单清洗。
4. 前端权限只控制体验，真正授权必须在服务端执行。
5. 下载和预览不要信任用户提供的任意 URL。

前端测试

优先覆盖：权限路由、请求拦截器、流式解析、关键表单映射、状态机转换和线上缺陷回归。组件快照不能替代业务断言；E2E 应覆盖“导入知识 -> 状态变化 -> 发起问答”和“线索创建 -> 关联 -> 筛选/导出”等关键闭环。

主 STAR：生产构建优化

Situation

公司运维系统在 k8s 干净容器中构建，无法复用开发机缓存。旧 `build:sre` 每次发版都会重新构建仓库内的 `@arco-design/web-vue`，即使只改普通业务页面，也重复执行完整组件库构建；流程慢，且源码与产物一致性依赖人工判断。

Task

作为项目唯一前端，在不牺牲产物正确性的前提下减少重复构建，并把组件源码与预构建产物的一致性变成自动校验规则。

Action

1. 拆分 `pnpm build:arco` 与 `pnpm build:sre`。
2. 只有 Arco 组件、样式、图标或构建脚本变化时才执行 `build:arco`。
3. 提交 `es`、`lib`、`dist` 和 `.build-manifest.json`，让干净容器直接消费预构建产物。
4. 遍历影响产物的源码输入，规范化路径与换行后计算 SHA-256。
5. `build:sre` 先检查必需产物、manifest 和当前源码 hash；不一致立即失败并提示重建。
6. 生产构建跳过非必要 SDK 类型声明 `rollup`。
7. 将 `dev/poc/pre/prod` 参数透传给 Vite `--mode`。
8. 编写打包部署说明，固化日常发版、组件库变更和异常处理流程。

Result

可以确认：普通业务发版不再执行整套 Arco 重建；源码变化但忘记更新产物时会在业务构建前失败；干净容器不依赖本地缓存；发布规则从人工记忆变成脚本校验和文档流程。

当前没有旧/新 CI 耗时数据，因此不说“提升 80%”。面试前从 CI 历史各取 5-10 次构建，比较中位数与 P95，再补真实数字。

方案取舍

为什么提交生成产物：当前构建环境无法稳定复用远程缓存，预构建产物适配现实约束。代价是仓库体积和源码/产物双提交，因此必须用 manifest 防止漂移。未来有可靠制品仓库或远程缓存后，可把产物迁出 Git。

追问速答

为什么 hash 排除 `es/lib/dist`？生成产物不能参与源码输入，否则形成循环变化。

为什么规范化 CRLF/LF？避免相同源码在不同系统上得到不同 hash。

manifest 被手改怎么办？当前能校验源码 hash 和必需产物；进一步可让 CI 独占生成、制品签名或对产物做二次 hash。

为什么不每次重建最安全？重复工作不自动带来安全；正确性来自可重复构建、输入校验、测试和制品治理。

Java / Spring Boot 深挖

网关信任边界

Gateway 负责验证 Cookie/JWT，解析用户和租户信息，再向下游传递受信 Header。下游不能接受公网直接访问后由客户端自行填写同名 Header，否则会造成身份伪造。生产环境应通过网络策略限制下游入口，并由下游校验内部签名或服务身份。

为什么业务服务仍要授权

网关回答“你是谁”，业务服务回答“你能访问哪条记录”。即使身份可信，也必须按租户、角色、智能体归属和资源关系做对象级授权，防止 IDOR。

external 为什么是适配层

爬虫、对象存储和外部知识服务的协议、错误码、超时和返回字段可能变化。适配层应统一内部 DTO、错误模型、超时与重试，并隔离供应商细节，避免业务 Controller 直接拼第三方请求。

HTTP 调用注意点

设置连接、读取和总体超时；只对幂等请求或带幂等键的请求重试；使用指数退避加 jitter；区分 4xx 永久错误与 5xx/超时临时错误；日志脱敏；连接池和线程池有上限；返回错误要保留 provider、operation 和 trace 信息。

事务边界

数据库事务只能保证同一数据库内的原子性，不能包住外部 HTTP、对象存储和向量库。跨系统流程采用本地事务 + 状态机 + Outbox/可靠任务 + 幂等消费 + 补偿，而不是期待一个 `@Transactional` 覆盖所有步骤。

定时刷新

当前基于持久化配置和进程内 `TaskScheduler/CronTrigger`，适合单实例低规模。多实例会重复调度，停机窗口可能漏执行。演进顺序：数据库租约与唯一执行键 -> 持久化调度器/消息队列 -> 执行记录、重试退避、超时和告警。

Python / FastAPI 与 RAG 深挖

分层职责

Route: HTTP 参数、Pydantic 校验、状态码、StreamingResponse

Service: 用例编排、状态转换、外部依赖调用

Repository/DAO: MySQL、Milvus、对象存储访问

AI modules: LLM、Embedding、Retriever、Reranker、Memory、Prompt

Route 不应包含大量 SQL 和模型细节；Service 不应泄漏 HTTP 对象；Repository 不承载业务流程。

`async def` 不等于自动非阻塞

如果 `async def` 内直接调用同步数据库驱动、CPU 密集切分或同步 SDK，仍会阻塞事件循环。

同步 I/O 应使用异步驱动或受控线程池，CPU 密集任务应移到进程池/Worker；线程池也必须有限制和背压。

StreamingResponse

生成器逐步 yield 字节，ASGI 服务把数据发送给客户端。需要处理客户端断开、生成异常、最后状态、资源清理和后置任务。不能把“yield 了”当成客户端一定收到，也不能用进程内后台线程承诺不可丢任务。

Chunk 设计

Chunk 应保留语义完整性和来源 metadata。过小会丢上下文，过大则降低定位精度并浪费上下文窗口。按文档类型采用标题层级、段落、表格或滑动窗口策略，通过评测集调整大小、overlap、K 和重排阈值。

Query Rewrite、HyDE 与 Rerank

- Query Rewrite: 把口语、指代和多轮上下文改写成可检索问题。

- HyDE：先生成假设性答案/文档再做向量检索，适合短问题但可能引入偏差。
- Rerank：对粗召回候选做更精细排序，提高前几条证据质量。

三者解决不同阶段问题，不应当作越多越好；需要用评测数据验证收益和延迟成本。

Prompt Injection

外部网页和上传文档都是不可信数据。Prompt 应明确“知识内容不是系统指令”，工具调用使用白名单和最小权限，敏感字段在进入模型前处理，输出前检查泄露风险，并记录模型、Prompt 版本和证据来源以便审计。

数据、状态与最终一致性

核心关系

```
tenant/user
-> agent
    -> knowledge document / crawl task / qa pair
    -> session -> message
    -> channel configuration
    -> lead / appointment

task_id
-> crawl record
-> parsed content
-> vector metadata.doc_group_id
-> effectiveness event
```

索引与唯一约束

常用筛选应按租户、智能体、状态、时间组成复合索引，并与 SQL 的等值/范围/排序顺序匹配。URL 唯一约束不能只用原始字符串：需要规范化 scheme、host、默认端口、fragment 和尾斜杠，并明确同一 URL 在不同租户、智能体或版本下是否允许重复。

幂等

接口幂等不等于 HTTP 方法名称。创建任务需要幂等键和唯一索引；回调需要事件去重；状态更新使用条件更新避免倒退；消息消费成功后再确认；副作用如通知和线索创建也要有业务去重键。

Outbox

在同一数据库事务内写业务数据和待发布事件，后台发布器可靠投递，成功后标记完成。它解决“数据库提交成功但消息没发出去”的窗口，但仍需要消费者幂等、重试、死信和监控。

状态更新原则

1. 状态只能沿允许路径前进，使用条件更新保护。
2. 每次转换记录时间、执行者、失败阶段和错误摘要。

3. 重试要区分新执行次数与同一业务任务。
4. `cancelled` 需要真正传播取消信号，不只是改数据库字段。
5. 终态回调重复到达应幂等返回，不再次触发副作用。

安全、性能与可观测性

主要信任边界

浏览器和渠道回调是不可信输入；Gateway 是外部入口；Java/Python 服务之间是内部但仍需认证；外部网页和文件是不可信内容；模型 Provider 是第三方数据处理边界；MySQL/Milvus/对象存储保存企业数据，必须按租户和智能体隔离。

SSRF 与文件安全

服务端抓取 URL 或代理图片时，应只允许 http/https，解析 DNS 后拒绝 loopback、私网、link-local 和云 metadata 地址；限制重定向并逐跳复验；使用域名 allowlist、出网代理和下载大小/超时限制。上传文件要校验扩展名、MIME 和魔数，随机化文件名，隔离存储，限制解压尺寸并按需做病毒扫描。

延迟拆解

总耗时 = 网关 + 业务查询 + Query Rewrite + Embedding
+ 向量检索 + Rerank + 模型首 Token + 持续生成 + 网络发送

首 Token 和完整回答时间是不同指标。优化前先按阶段打点，查看 P50/P95/P99；平均值会掩盖长尾。

超时、重试、熔断与背压

每层超时必须小于上游剩余预算；只重试临时且幂等的错误；指数退避加 jitter；Provider 连续失败时熔断并半开探测；按租户和 Provider 隔离并发；队列设置上限并返回排队或限流信息，不能无限堆积。

关键观测指标

链路	指标
知识摄入	提交量、各阶段成功率、失败原因、端到端生效 P50/P95、重试次数
WebGPT 爬虫	队列深度、等待时长、抓取策略分布、页面成功率、超时率、域名限速、父子文档失败数
RAG	召回耗时、Rerank 耗时、首 Token、总耗时、中断率、拒答率、引用覆盖
Provider	请求量、错误率、限流率、超时率、token/费用、熔断状态
AP 集成	内部接口可用率/延迟、配置版本、配额同步差异、预警触发/去重/通知成功率
前端	页面错误、接口失败、流中断、关键交互耗时、版本和环境
业务	活跃用户、问答量、解决率、线索有效率、转人工率

当前项目约 8 个用户，更应先建立基线和正确口径。没有监控数据时，可以描述“会怎么测”，不能声称已经达到某个 SLO。

工程质量、技术债与演进

测试金字塔

- 1. 单元测试：状态机、URL 规范化、权限判断、流式事件解析、Prompt 组装。
- 2. 集成测试：Controller/Route + 数据库、Milvus Adapter、对象存储和 Provider mock。
- 3. 契约测试：前端 DTO、Java 外部适配、FastAPI 响应事件和错误码。
- 4. E2E：知识导入到问答、渠道回调到消息、线索创建到导出。

发布完成标准

代码通过 lint/typecheck/受影响测试；数据库变更可回滚；配置只通过变量名和示例表达；关键指标和告警就绪；灰度范围明确；回滚步骤可执行；上线后验证核心链路和日志；变更记录包含风险与观察窗口。

技术债优先级

优先级	问题	理由
P0	越权、SSRF、敏感信息泄露、回调伪造	安全和数据影响不可接受
P0	爬虫关闭 TLS 校验或原始凭证长期进入 Redis	可能造成中间人攻击和凭证扩散
P1	不可丢任务仍依赖进程内线程、跨服务无统一 trace、失败不可定位	直接影响可靠交付
P1	WebGPT Redis 状态缺失被解释为完成、终态回调仅有限重试	可能产生伪成功和知识状态永久不一致
P1	RAG 没有离线评测和线上质量口径	无法判断模型/检索调整是否变好
P2	状态机、幂等、取消和重试不完整	规模增长后重复和脏状态增多
P2	API 契约分散、前端映射易漂移	增加联调与线上回归成本

优先级	问题	理由
P3	模块进一步抽象、代码风格统一	在正确性和可靠性之后处理

十倍规模演进

API 与 Worker 分离；任务进入持久化队列；按租户/类型分区和限流；对象存储承接中间文件；批量 Embedding；Milvus 分区或过滤字段优化；Provider 统一 Adapter 与配额；全链路 trace；死信和人工补偿；容量压测以队列等待、首 Token、CPU、内存和 Provider 限额为核心。

高频技术追问 37 题

1. 你最熟的项目部分是什么？

三套前端应用和生产构建发布是我主导范围；知识摄入和 RAG 是我通过 AI 辅助开发、接口联调和源码阅读掌握的跨栈链路。我会先讲构建优化，再按追问进入 Java/Python。

2. 为什么前后端分成 Java 与 Python？

按能力生态和变化率划分：Java 承接稳定业务、权限和适配；Python 承接 AI 生态。代价是契约、追踪和一致性，需要统一治理。

3. 为什么 URL 导入是异步任务？

爬虫、解析、Embedding 和向量写入耗时不可控且失败点多，不能长期占用 HTTP。异步化后必须补状态、幂等、重试和可观测。

4. task_id 有什么作用？

关联一次知识任务在任务表、内容、向量 metadata 和生效事件中的记录，也用于排障；它不是授权凭证。

5. 导入成功但问答搜不到，怎么查？

按 `task_id` 检查提交、爬虫、内容、解析、chunk、Milvus metadata、生效状态，再检查 QA 的服务端过滤范围、K、阈值和 Rerank。

6. 如何防止重复导入？

规范化 URL，结合租户、智能体和版本形成业务键；创建接口支持幂等键；数据库唯一索引兜底；重复回调按事件 ID 去重。

7. 为什么不能只靠前端传 `task_ids`?

前端输入不可信，会造成跨租户或跨智能体检索。服务端必须根据身份和资源关系计算允许范围。

8. POST 流式响应为什么不用 EventSource?

提问需要复杂请求体，POST 更自然；使用 fetch 读取流。代价是重连、取消和事件解析由应用实现。

9. 流式内容为什么会乱码或 JSON 解析失败?

网络 chunk 不等于字符或协议事件边界。必须用流式 TextDecoder 和残留 buffer，完整分隔后再解析。

10. 如何真正停止生成?

前端 AbortController 取消连接，服务端检测断开并传播 cancellation，Provider 支持时取消上游；长任务保存进程/请求句柄并做条件状态更新。

11. `async def` 为什么仍可能阻塞?

其中调用同步数据库、同步 SDK 或 CPU 密集逻辑会阻塞事件循环。需要异步驱动、受控线程池、进程池或 Worker。

12. 数据库和向量库如何一致?

无法使用单个事务，采用状态机、Outbox/任务表、幂等写入、重试和补偿；只有向量可检索后才更新 effective。

13. 重试会带来什么风险？

放大故障、重复副作用和重试风暴。只重试临时错误，限制次数，指数退避加 jitter，并要求幂等。

14. 什么情况下使用消息队列？

当任务不可丢、执行时间长、需要削峰、重试、独立扩容或跨服务解耦时。低规模先用任务表也可以，但必须承认可靠性边界。

15. RAG 的 K 越大越好吗？

不是。K 大会增加噪声、上下文成本和延迟。先保证召回，再用 Rerank 和阈值控制精度，通过评测集调参。

16. 如何衡量 RAG 好坏？

分检索、生成、拒答、体验和业务五层；离线固定评测集，线上按版本做小流量对比，不能只看主观案例。

17. 如何降低 Prompt Injection 风险？

把外部内容视为数据，系统指令与知识隔离；工具白名单和最小权限；敏感字段处理；输出审查；记录证据和版本。

18. Gateway 鉴权后下游为什么还要校验？

Gateway 认证身份，下游做资源授权；同时限制下游网络入口，防止伪造受信 Header。

19. Cookie、JWT、localStorage 怎么选？

浏览器会话优先考虑 Secure/HttpOnly/SameSite Cookie 并配合 CSRF；跨端 API 可用短期 Bearer Token。长期敏感 Token 不放 localStorage。

20. 如何防 SSRF?

协议白名单、DNS 解析后拦截私网/loopback/metadata、限制重定向并逐跳复验、域名 allowlist、出网隔离、大小与超时限制。

21. 如何设计索引?

从真实 SQL 的等值过滤、范围和排序出发；常见是租户/智能体 + 状态 + 时间。用 EXPLAIN 验证选择性和回表成本。

22. 为什么平均耗时不够?

平均值会掩盖长尾。用户体验和超时风险主要看 P50/P95/P99，并按阶段、Provider、模型和渠道拆分。

23. 如何保证 AI 生成代码正确?

先定义契约和失败条件；限制改动范围；逐文件审查 diff；补成功、失败、重复、权限和边界测试；运行受影响模块验证；无法解释的代码不提交。

24. 当前项目最大不足是什么?

线上指标和 RAG 评测不足；部分长任务可靠性仍需加强；跨服务 trace、对象级授权和错误模型需要统一。这些比继续增加框架更优先。

25. 如果重做，你先改什么?

先建立事实基线和 trace；再把不可丢后置任务迁到持久化队列；最后补 RAG 评测与权限过滤的自动化门禁。

26. AP 和 Autobots 的边界是什么？

AP 是平台控制面，提供租户、Agent、模型供应商、文档、配额和观测能力；Autobots 是面向客服业务的产品面，负责知识编排、RAG、会话、线索、渠道和运营页面。边界不是按调用次数划分，而是按数据所有权、变化率和资源模型划分。Autobots 可以适配和展示 AP 能力，但不应复制一套租户配额真相。

27. 为什么不把爬虫直接放进 `autobots-ai`？

网页抓取面对不可信外部站点，需要 URL 限制、浏览器资源隔离、策略选择和任务调度；AI 服务关注解析、Embedding、检索和模型调用。独立 WebGPT 可以单独限速、扩容、重试和排障，降低爬虫对在线问答的影响。代价是要治理任务状态、文档契约、回调鉴权和最终一致性。

28. AP 不可用时，聊天能不能继续？

要先区分旁路能力和主链路。流量预警属于旁路，当前可以 fail-open，记录日志并用 Java 本地逻辑降级；认证、租户授权、模型实际扣费和知识权限属于主链路，不能为了可用而放行。长期应有缓存、熔断、半开、配置版本和明确的降级提示。

29. `monthlyTokens`、`monthlyApiCalls` 和 `maxConversationTurns` 怎么区分？

`monthlyTokens` 是租户本月 Token 配额，`monthlyApiCalls` 是本月对话调用/轮次配额，`maxConversationTurns` 是单个会话最多允许的轮次。它们的时间窗口和业务含义不同，不能把单会话限制累加成月配额，也不能把 -1 的无限语义当成 0。

30. WebGPT 为什么要先建根文档占位？

根页面和子页面是并行抓取的，子页面写 AP 文档时需要 `parentId`。提前创建占位并把 `parent_doc_id` 放入 Redis，可以让子页面建立稳定层级；根正文抓到后再更新占位。它解决的是父子关系竞态，不代表内容已经抓取成功，更不代表知识已经生效。

31. WebGPT 如何选择 Requests、SessionPage 和 Playwright?

先用 Requests 走低成本路径；当静态结果发现的链接不足时，再并行比较不同策略的链接和内容质量，按结果选择并缓存。这个方案是渐进增强，但要关注每页启动浏览器的资源成本、策略缓存失效、页面反爬和域名限速。面试中要同时讲收益和代价。

32. Redis 中任务状态消失意味着什么?

不能直接判断成功。状态可能是完成后清理、过期、取消清理、Redis 故障或任务丢失。当前实现存在把缺失状态返回为 completed 的语义风险；更可靠的设计是任务终态写持久化表，Redis 只做实时进度和队列，查询时区分 `not_found/expired/completed/cancelled`。

33. 如何保证 WebGPT 回调只处理一次?

以真实 crawler `task_id` 加事件类型或事件 ID 建唯一键，消费者使用条件更新：只有 `pending/processing` 才能转 `effective/failed`，已是终态的重复回调返回幂等成功，不再次触发通知或统计。有限重试解决短暂网络错误，Outbox/DLQ 解决最终丢失问题。

34. 爬虫里的 Token 为什么是安全问题?

任务调度需要携带调用 AP 的身份上下文，如果把原始用户 Token 长时间放在 Redis，可能被日志、运维或越权读取，且重试会延长暴露窗口。应使用短期、最小权限、按租户隔离的凭证引用，Redis 只保存引用和过期时间，Worker 通过受控凭证服务换取一次性调用凭证，并对敏感字段脱敏。

35. 如何防止爬虫变成 SSRF?

只允许 http/https 只是第一步。需要规范化 URL，解析 DNS 后拒绝 loopback、私网、link-local 和云 metadata 地址；每次重定向后重新检查；限制允许域名、深度、页数、响应大小、总耗时和并发；出网走隔离代理并记录审计。域名 allowlist 不能替代 IP 级校验。

36. “抓取完成”和“知识生效”有什么区别？

抓取完成表示 WebGPT 已经处理根/子页面并更新抓取终态；知识生效还需要 AP 文档写入、Autobots 解析、切分、Embedding、Milvus 写入，并通过可检索性校验。两者必须有不同状态或至少不同阶段时间戳，否则前端会显示成功但用户搜不到。

37. 这部分代码主要是 AI 写的，你如何证明是你的能力？

我不把生成代码等同于手写经验。我的责任是把需求拆成契约、状态和失败边界，审查每个 diff，补成功/失败/重复/权限测试，完成跨服务联调，能按 `task_id` 或 `trace_id` 定位问题，并对上线结果负责。当前前端和构建优化是我的主导经验；AP 与 WebGPT 是源码理解和集成能力，我会明确这个边界。

系统设计 7 题

场景一：设计可靠的知识摄入平台

API 接收请求并写任务表与 Outbox；队列按任务类型分发；Worker 分阶段执行爬取、解析、Embedding 和向量写入；每步保存状态与幂等键；失败指数退避，超过上限进死信；对象存储保存原文和中间结果；最终通过检索探针确认生效；用户按 task ID 查询或订阅状态。

场景二：设计多租户 RAG

所有知识带 tenant/agent/visibility metadata；权限服务计算可访问范围；检索层强制过滤，不能由客户端绕过；Embedding 与向量集合按规模选择共享+过滤或分区；日志和评测数据脱敏；缓存键包含租户和权限版本；建立跨租户泄露测试。

场景三：Provider 故障如何降级

统一 Provider Adapter；按能力、健康、配额、成本和质量路由；设置分层超时、有限重试和熔断；有等价模型时切换并标记差异；无等价能力时返回可解释错误或转人工；记录模型版本、费用、失败原因和 trace。

场景四：问答量增长 10 倍

入口限流；按租户配额；API 与生成 Worker 分离；流式连接设置上限；Embedding/检索连接池容量规划；Prompt 和常见问题按权限安全缓存；模型并发舱壁；队列展示排队位置；压测首 Token、完整耗时和断线率。

场景五：设计大文件上传与解析

客户端申请对象存储分片凭证，分片并发和断点续传；服务端登记对象和 hash；异步做病毒扫描、格式检查和解析；限制解压尺寸与页数；任务状态可查询；下载使用短期签名 URL；权限在签发时校验。

场景六：设计 AP 配额与流量预警一致性

把 AP 作为租户配额、平台用量和预警配置的真相源，Autobots 只负责业务用量补充、触发时机和页面展示。每次问答完成后用事件或可靠调用上报用量，按租户/月/指标/阈值做去重；AP 查询失败时旁路预警 fail-open，但充值、扣费和授权必须 fail-closed；通知写历史前后都用业务键幂等。进一步加入配置版本、对账任务、熔断和最后一次配置缓存，并把“未检查”与“未超阈值”区分开。

场景七：设计可恢复的 WebGPT 爬虫平台

API 只负责校验、规范化、幂等任务落库和入队；Redis 做短期进度与公平调度，持久化任务表记录终态、重试和错误阶段；Worker 按域名限速，先静态抓取再按证据升级浏览器策略；根文档占位确保父子关系，页面推送以 URL/版本幂等；回调用 Outbox/DLQ 保证知识生效事件最终送达；取消和超时都要传播到 HTTP 与浏览器；权限、凭证、SSRF 和下载大小在边界处强校验。

STAR、简历与行为面

STAR 一：生产构建优化 [主导]

S：干净 k8s 容器每次发版重建 Arco，重复耗时且一致性靠人工。

T：减少重复构建，同时保证源码与产物可自动验证。

A：拆分构建命令、提交预构建产物、计算规范化源码 SHA-256、写 manifest、业务构建前校验、透传环境 mode、补发布文档。

R：普通业务发版跳过 Arco 重建；源码与产物漂移提前失败；发布规则可执行。耗时百分比待 CI 历史核验。

STAR 二：知识摄入与问答解耦 [AI 辅助/掌握]

S：URL 导入跨越 Autobots、WebGPT、AP 文档和知识生效回调，实时问答不能等待爬虫、解析和向量化。

T：前端正确呈现异步状态，并完成跨 Java/Python 的联调和失败定位。

A：围绕真实 crawler `task_id` 串联 external 提交、Redis 任务、根文档占位、父子文档、knowledge-effectiveness 回调和向量 metadata；问答只使用服务端计算出的已生效知识范围；验证处理中、失败、重试和流式问答。

R：形成可按 task ID 定位的端到端链路。该故事证明跨栈理解和验收，不包装成个人独立后端架构成果。

STAR 三：AP 配额与预警联调 [AI 辅助/掌握，参与度需确认]

S：问答用量、租户配额、预警配置和日报字段分布在 AP 与 Autobots，两边字段别名和失败策略不一致会造成误告警或阻断聊天。

T：明确数据真相源和字段语义，保证预警异常不影响主问答，并让页面展示与通知使用同一口径。

A：核对 `monthlyTokens`、`monthlyApiCalls` 和 `maxConversationTurns`；沿 Python client、Java `TrafficAlertAutoService` 与 AP internal API 跟踪调用；验证 fail-open、本地回退、Redis 去重和日报字段归属；建议用契约测试收敛字段别名。

R：形成可解释的跨系统数据边界和排障路径。只有你确实参与过这次联调时才以 STAR 讲；否则把它放在“源码理解与演进建议”。

STAR 四：线索产品与人员关联 [待补]

只在补齐故障表现、影响、根因、具体改动、验证和结果后使用。当前可安全说明自己负责前端链路和跨角色联调，不写虚构指标。

简历项目描述

1. 作为项目唯一前端，负责 Vue 3 + TypeScript 技术栈下 C 端聊天、企业管理端和平台管理端三套应用，覆盖智能体、知识库、对话、线索、数据看板、运营配置及前端发布交付。
2. 推动生产构建优化，将每次发版重复构建 Arco 调整为预构建复用，通过 SHA-256 源码 hash 和构建清单校验产物一致性，使普通业务发版跳过组件库重建并适配干净 k8s 构建环境。
3. 使用 AI 编程辅助参与 Java/Python 跨栈需求，围绕 Autobots、AP 和 WebGPT 的网页知识摄入、Agent/模型配置、配额预警与 RAG 问答完成需求拆解、接口联调、测试验证和问题定位。
4. 沿真实 crawler `task_id` 打通 URL 提交、Redis 任务、父子文档写入、知识生效回调与前端状态展示，能够解释异步状态、幂等、失败降级和 POST 流式响应；仅在参与度和结果可核实时放入正式简历。

为什么转全栈

我不是为了离开前端，而是把交付责任扩展到浏览器之外。知识页面显示“处理中”，真正的体验取决于后端任务是否幂等、失败是否可见、向量何时可检索；构建优化也需要理解 monorepo、产物和容器环境。我的路径是保留前端深度，同时把接口、数据、后端可靠性和 AI 评测补成第二能力曲线。

AI 写的代码算不算经验

AI 是开发工具，但进入项目后的责任不能交给 AI。前端和构建优化是我的直接主导经验；Java/Python 目前主要是 AI 辅助，我负责需求、边界、diff 审查、联调和验证，只讲能解释的部分。我不会把它等同于多年手写后端经验；下一步是从“借助 AI 跨栈交付”提升到“独立设计、审查和维护后端模块”。

38 岁转型的岗位定位

优先投递“前端偏全栈、AI 应用工程师、企业应用交付、前端负责人兼跨栈”岗位。核心价值是成熟交付、业务理解、前端深度和 AI 提效；纯 Java/Python 后端岗位会直接与多年后端候选人比较，现阶段证据不足。

面试目标不是证明“什么都会”，而是证明：我能独立守住前端主战场；能借助 AI 穿透接口、数据和后端链路；知道 AI 代码的验证责任；能把模糊需求落成可发布、可回滚、可排障的系统。

转型成功率：条件区间，不是行业统计

不存在可信的“38 岁前端转全栈统一成功率”。城市、学历、薪资预期、空窗期、岗位数量和投递质量都会改变结果。下面是基于当前项目证据的个人估算：把“成功”定义为 **6-12 个月内进入目标岗位并通过试用期**，前提是精准投递、薪资预期不逆势大涨，并持续补强后端证据。

目标岗位	当前证据下的主观区间	判断
前端主导的 AI 应用 / 企业应用交付	45%–60%	最匹配；能复用多年工程经验、三段负责人经历和 AI 项目链路
前端偏全栈	30%–45%	可行；后端独立设计、测试和上线证据仍偏少
通用全栈，要求独立负责 Java/Python	20%–30%	会被追问事务、鉴权、并发、任务可靠性和生产排障
纯 Java/Python 中高级后端	10%–20%	不建议作为主赛道；会直接与多年后端经历竞争

如果在 3–6 个月内补齐“一段独立 Java 纵切、一段独立 Python/RAG 模块、自动化测试、部署与监控、两个真实指标”，前端偏全栈 / AI 应用方向的个人估算可提高到 **60%–75%**。这仍然是决策区间，不是承诺。

38 岁的真实劣势是：企业会担心薪资、稳定性、学习速度和转型后的产出确定性。对应策略不是隐藏年龄，而是用证据降低风险：第一周能接前端主任务；一个月能独立完成小型跨栈纵切；能审查 AI 代码；遇到线上问题能止损、定位、回滚和复盘。

市场方向支持“AI 应用 + 软件交付”，但也说明技能迭代更快。国务院 2025 年“人工智能+”意见提出推动软件信息服务企业智能化转型、培育 AI 应用服务商，同时强调技能培训与就业风险评估；PwC 2025 全球 AI 就业报告基于近十亿条招聘广告，报告 AI 暴露岗位的技能变化速度快 66%、同岗位 AI 技能工资溢价 56%。这些是方向信号，不能直接换算成个人 offer 概率。

来源（检索日期 2026-08-07）：国务院《关于深入实施“人工智能+”行动的意见》（2025-08-21）；PwC《2025 Global AI Jobs Barometer》；工业和信息化部教育与考试中心《生成式人工智能系统测试员国家职业标准》初审信息（2026-02-05）。

行为面速答

需求不清楚： 先写用户目标、输入输出、成功标准和异常边界，与产品和上下游确认；再做可演示的垂直切片。

技术分歧： 把争议转成约束和指标，例如交付时间、正确性、成本、兼容性和维护成本；能验证就做小实验，决定后记录并共同执行。

线上问题： 先止损和界定影响，再保留证据；按 trace/task/session 还原链路；修复后补监控、测试和操作手册。

不会的问题： 先澄清约束，说明已知与未知，给出推导和验证办法；不编 API，不把建议说成现状。

红线与源码细节

不要这样说

1. “我负责整个 Java/Python 后端。” 真实情况是 AI 辅助开发与联调。
2. “项目有高并发经验。” 当前公开用户约 8 个，没有可靠 QPS。
3. “构建提升 80%。” 还没有 CI 基线。
4. “用了 Milvus 就不会幻觉。” 向量检索只是 RAG 的一个环节。
5. “task ID 就能保证权限。” 它只是关联标识。
6. “SSE 会自动重连。” 当前是 POST fetch 流，需要自行重连和取消。
7. “进程内线程是可靠任务系统。” 实例崩溃会丢后置任务。
8. “微服务天然更快。” 服务拆分主要解决职责、演进和故障边界。
9. “AI 生成代码所以开发能力已经等同资深后端。” 工具效率和后端基本功是两件事。
10. “AP 和 WebGPT 都是我主导设计的。” 当前能证明的是关联项目源码理解、AI 辅助和联调能力，除非另有真实记录。
11. “WebGPT 抓取完成就代表 RAG 已生效。” 还需要文档写入、解析、向量化和检索可见性确认。
12. “Redis 中查不到任务就是 completed。” 缺失也可能来自过期、取消、故障或丢失。

更专业的表达

- “前端与构建是我的主导范围；Java/Python 是 AI 辅助交付，我能解释、验证和排障。”
- “当前规模不大，先保证正确性和可观测；十倍规模时再按资源边界拆 Worker。”
- “项目已具备记录指标的代码和事件，线上基线仍需统一统计口径。”
- “当前使用任务表和进程内机制打通流程，不可丢任务会演进到持久化队列。”
- “RAG 效果用检索、生成、拒答和业务指标共同评估。”
- “AP 是平台配置和配额真相源；Autobots 做客服业务编排；WebGPT 是独立抓取执行面。”
- “预警旁路可以 fail-open，但认证、权限和扣费不能沿用同一降级策略。”

面试官可能追的源码细节

1. 为什么知识生效回调可能返回 409：覆盖 pending 记录尚未落库的短暂竞态。
2. 为什么停止 UI 不等于取消模型：必须用 AbortController 并把取消传播到服务端和 Provider。
3. 为什么 task_id 不能由前端决定权限：客户端输入不可信，服务端必须按关系计算范围。
4. 为什么 CRLF/LF 会影响构建 hash：字节不同会产生不同摘要，需要规范化输入。
5. 为什么预构建产物要有 manifest：解决源码与生成物双提交的漂移风险。
6. 为什么 WebGPT 先创建根文档占位：避免子页面先完成却拿不到 parentId。
7. 为什么知识生效使用爬虫真实 task_id：请求标识和实际任务标识可能不同，事件必须跟随真实执行。
8. 为什么回调 409 可以重试：pending 事件可能暂时不可见，是状态竞态而非永久参数错误。
9. 为什么 AP 预警失败不阻断聊天：预警是旁路能力；但必须记录“未检查”，不能伪装成“未超限”。
10. 为什么 maxConversationTurns 不能当月配额：它限制单会话，而 monthlyApiCalls 才是月度调用口径。

自检、训练与口袋卡

面试前必须能脱稿

- 1. 用 30 秒和 90 秒讲清项目。
- 2. 画出 AP、Autobots、WebGPT 总体架构，以及 URL 摄入和 RAG 问答两条链路。
- 3. 完整讲生产构建优化 STAR 和取舍。
- 4. 明确说出哪些是主导、AI 辅助和掌握。
- 5. 回答 task ID、父子文档、幂等、流式解析、取消、配额归属、RAG 评测、SSRF 和最终一致性。
- 6. 说出三项优点、三项不足和演进顺序。
- 7. 每个简历数字都知道来源和口径。

七天训练

天	训练重点	产出
Day 1	个人事实、30/90 秒介绍、贡献边界	录音 2 次，删掉夸大表述
Day 2	前端三应用、状态、流式 UI	画页面/API/状态图
Day 3	构建优化主 STAR	讲清输入、hash、产物、失败保护和取舍
Day 4	WebGPT URL 摄入、Redis、父子文档、状态机和幂等	画时序图并排查 5 个故障
Day 5	RAG、DeepSeek、检索、评测和安全	回答 10 个 AI 问题
Day 6	AP 配额预警、Java/Python 基础与 7 个系统设计	限时 10 分钟/题
Day 7	60 分钟模拟面试	只补暴露出的三项短板

10 分钟模拟脚本

1. 1 分钟：项目介绍和个人职责。
2. 2 分钟：画 AP、Autobots、WebGPT 总体架构与数据边界。
3. 3 分钟：讲构建优化 STAR。
4. 2 分钟：讲 URL 摄入到 RAG 问答。
5. 1 分钟：承认一个技术债并给治理顺序。
6. 1 分钟：提出两个高质量反问。

面试当天口袋卡

定位：企业智能客服与 AI 应用平台

闭环：智能体 -> 知识 -> RAG 问答 -> 会话 -> 线索/运营

三系统：AP 控制面 / Autobots 业务面 / WebGPT 抓取执行面

主责：三套 Vue 应用 + 前端构建发布

主 STAR：Arco 预构建复用 + SHA-256 manifest

跨栈：AI 辅助 Java/Python，负责需求、审查、联调、验证

模型：主问答 DeepSeek

规模：约 8 个用户，不虚构高并发

三个 ID: trace_id / task_id / session_id

四个边界：提交 != 生效；抓取完成 != 知识生效；task_id != 权限；流停止 != 服务端取消

三个演进：持久化任务终态 / 统一 trace / RAG 评测与授权

反问面试官

1. 这个岗位更偏端到端业务交付，还是长期负责某个后端服务？
2. 团队对 AI 代码的评审、测试、数据合规和效果评测如何落地？
3. 当前最需要解决的是交付效率、稳定性、数据一致性还是 AI 效果？
4. 前后端和算法的接口契约、代码评审由谁负责？
5. 入职三个月最重要的可验收结果是什么？

附录

关键代码入口

主题	入口
前端路由	<code>autobots-frontend/app*/src/router/index.ts</code>
URL 导入 UI	<code>app-manager/src/views/Knowledge/Content/Add/UrlUploadModal.vue</code>
知识 API	<code>app-manager/src/api/knowledge2.ts</code>
知识状态列表	<code>app-manager/src/views/Knowledge/Content/index.vue</code>
定时刷新 UI	<code>ScheduleSettingsModal.vue</code>
前端生产构建	<code>autobots-frontend/scripts/build-sre.mjs</code> 、 <code>build-arco.mjs</code>
Gateway 鉴权	<code>autobots-gateway</code> 中 Auth Filter / route config
Java 知识适配	<code>autobots-external knowledge Controller/Service</code>
FastAPI 启动	<code>autobots-ai/start.py</code>
文档摄入	<code>autobots-ai/server/ingest_route.py</code> 、 <code>document_route.py</code>
流式问答	<code>autobots-ai/server/qa_route.py</code>
RAG 能力	<code>autobots-ai/ai/</code> 下 Retriever、Reranker、LLM、Memory
线索	<code>lead_route.py</code> 、 <code>lead_form_route.py</code> 与 user 侧线索模块
AP 总览	<code>PROJECT_NAVIGATOR.md</code> 的 <code>agplatform</code> 部分、 <code>../agplatform/core</code>
AP 租户配额	<code>../agplatform/core/platform-tenant/.../InternalTenantQuotaController.java</code>
AP 流量预警	<code>../agplatform/core/platform-observe/.../InternalTrafficAlertController.java</code>

主题	入口
Autobots AP 客户端	<code>autobots-ai/services/</code> <code>agentic_platform_client.py</code> 、 <code>autobots-</code> <code>user/.../TrafficAlertPlatformClient.java</code>
预警业务编排	<code>autobots-user/.../</code> <code>TrafficAlertAutoService.java</code> 、 <code>TrafficAlertConfigService.java</code>
WebGPT API	<code>../webgpt_server/server.py</code> 的 <code>/crawl</code> 、 <code>/status/{task_id}</code> 、 <code>/cancel/{task_id}</code>
WebGPT 调度	<code>../webgpt_server/utils/</code> <code>redis_queue_manager.py</code>
WebGPT Worker	<code>../webgpt_server/crawler_worker.py</code>
AP 文档推送/回调	<code>../webgpt_server/utils/autobots_client.py</code> 、 <code>tests/</code> <code>test_knowledge_effectiveness_callback.py</code>

术语速查

术语	简洁解释
幂等	同一业务请求执行多次，最终效果与一次一致
Outbox	业务数据与待发布事件同事务写入，再可靠投递
最终一致性	多个系统不同时提交，但通过重试和补偿最终达到一致
熔断	依赖持续失败时快速拒绝，恢复后半开探测
舱壁	按租户或依赖隔离资源，避免单点故障拖垮全局
背压	下游处理不过来时限制上游生产速度
SSRF	攻击者诱导服务端访问内网或敏感地址
RAG	先检索外部证据，再让模型基于证据生成
Rerank	对粗召回候选做精细重排

术语	简洁解释
首 Token	请求开始到用户收到首个有效生成内容的时间
SLO	团队承诺并监控的可用性、延迟或成功率目标
Control Plane	管理配置、策略、租户和配额的控制面，不直接等同在线执行链路
Fail-open	依赖异常时放行主流程，适合非关键旁路；认证和授权通常不能使用
DLQ	多次处理失败后保存消息的死信队列，用于告警、排查和人工补偿

个人信息补全页

面试前填写并删除无法证明的模板：

- 构建优化前 5-10 次 CI 中位数/P95：【】
- 构建优化后 5-10 次 CI 中位数/P95：【】
- 线索关联问题的具体表现：【】
- 线索关联问题影响的角色/记录：【】
- 线索关联问题根因与修改文件：【】
- 线索关联问题回归场景与最终结果：【】
- 一个你独立补写的 Java/Python 测试：【】
- 一个你能脱离 AI 独立修改的后端模块：【】
- 一次你真实参与的 AP 或 WebGPT 联调任务、修改文件和验证结果：【】
- 一组 AP 配额/预警字段的线上真实口径与数据来源：【】
- 一次 WebGPT 任务的提交、抓取、文档、回调和生效时间戳：【】
- 知识量、日问答量、首 Token P50/P95：【】
- 一次失败、承担动作和流程改进：【】

补全优先级

1. P0：线索关联问题的故障、根因、动作、验证和结果；这是第二个真实 STAR 的基础。
2. P0：构建优化前后 CI 数据；至少各取 5 次，统一环境和命令后比较中位数与 P95。
3. P1：独立补写一个 Java 或 Python 测试，并能解释为什么覆盖成功、失败、权限和重复请求。
4. P1：从日志或事件表建立知识生效与首 Token 基线；没有权限拿数据时，准备清晰的采集方案。
5. P1：用一个真实 `task_id` 完成 WebGPT 到 AP 文档再到知识生效的全链路演练，并记录每个阶段的输入、状态和失败点。

完成标准

1. 每个简历数字都有来源、时间窗口、计算方式和可接受的追问。
2. 每个 STAR 都能说出自己修改过的入口、异常分支和验证动作。
3. AI 辅助代码能脱离提示词解释数据流、失败边界和测试结果。
4. 无法证明的内容留在建议方案，不进入“我做到了”的结果陈述。